Round 1 Technical Assessment

Purpose:

This assessment tests ML engineering judgement for a regulated, auditable scoring engine. We care more about safety, validation, explainability, auditability, and communication than leaderboard-style model accuracy.

Context

At Arbix AI, Yogyank is the scoring engine behind SaakhSetu. Yogyank does not output a Probability of Default. It outputs a bank-agnostic Entitlement Score. Bank-specific cutoffs, grade mappings, and eligibility decisions are handled outside the model in a versioned policy layer.

Because this score can influence a farmer's access to formal credit, the model pipeline must be mathematically sound, auditable, deterministic, and designed with explicit fairness checks, bias monitoring, and documented limitations.

Scenario

A junior data scientist recently drafted a baseline training script, broken_yogyank_training.py, to predict the Entitlement Score using farmer_scoring_sample_yogyank_round1.csv. They proudly reported an extremely high validation R^2 score.

However, upon initial review, we suspect the script contains critical MLOps, mathematical, and architectural flaws that make it dangerous to rely on.

Your task

You have exactly 90 minutes to audit the script and improve it. We are not expecting a perfect production system in 90 minutes. We are evaluating your judgement, prioritization, and ability to explain risk clearly.

1. Audit the code: assess whether the code is safe for real future scoring, including feature availability, validation design, encoding choices, artifact handling, and separation between model logic and policy logic.
2. Fix the flaws: rewrite the script into a safer, audit-conscious baseline suitable for review, not full production deployment.
3. Ensure auditability: save the model and preprocessing artifacts needed for reproducible scoring. Include a simple version stamp and a way to produce top 3 reason codes for a farmer score.
4. Write the audit memo: explain in plain English why the original script was dangerous, how your changes reduce risk, and what limitations remain.

Inputs provided

- farmer_scoring_sample_yogyank_round1.csv - synthetic sample dataset for the assessment.
- broken_yogyank_training.py - intentionally flawed baseline training script.
- This instruction sheet.

Required deliverables

- fixed_yogyank_training.py
- audit_memo.md. Use the structure specified in “Suggested audit memo structure” at the bottom.
- README.md with setup/run steps, assumptions, what you fixed, what you skipped due to time, and validation approach.
- LLM_NOTES.md if AI/coding tools were used. Use the structure below.
- Optional artifacts/ folder containing saved pipeline/model/encoder/schema/version files.

AI / LLM tool policy

Allowed with disclosure: Google, official documentation, open-source libraries, ChatGPT, Claude, Copilot, Cursor, and similar tools are allowed. We do not penalize responsible tool use. We do penalize hidden usage, inability to explain submitted work, and unsafe acceptance of tool output.

- You must disclose AI/coding-tool usage in LLM_NOTES.md if used.
- You must be able to explain and modify your submitted solution live in Round 2.
- Do not submit tool output that you cannot verify or defend.
- Help from another person, outsourcing, or hiding tool usage is not allowed.

Required LLM_NOTES.md structure, if tools were used

- Tools used: ChatGPT / Claude / Copilot / Cursor / other.
- Where used: flaw identification, code refactoring, debugging, tests, memo drafting, or other.
- Representative prompts: include 2 to 5 prompts, paraphrased if needed.
- What you accepted: list 2 suggestions you used.
- What you rejected or corrected: list at least 1 suggestion that was wrong, unsafe, incomplete, or not aligned with the assessment.
- What you personally verified: leakage check, OOT split, encoder boundary, saved artifacts, reason-code logic, and run output.

Submission rules

- **Create a fresh private GitHub repository.**
- **Make your first commit at the chosen start time and final commit near the end of the 90-minute window.**
- **Use meaningful commits. Avoid one large dump commit at the end.**
- **Push the code and share one GitHub repository link only.**
- **Your README should state start time, end time, approximate time spent, completed work, skipped items, and why.**

What we value

- Finding and fixing data leakage.
- Validation that realistically simulates future scoring.
- Separation of model score from policy logic.
- Leakage-safe encoding and frozen preprocessing artifacts.
- Clear reason codes and audit/versioning discipline.

- Founder-friendly explanation of risks and tradeoffs.
- Responsible AI-tool use with personal verification and clear ownership.

What we do not value

- Maximizing R^2 without questioning whether the result is believable.
- Mixing business policy into the target or hidden score adjustments.
- Fitting preprocessing on validation/scoring data.
- Only saving a model file without schema, feature list, encoders, or version metadata.
- Complexity without clarity.
- Submitting LLM-generated code or text that you cannot explain.

Suggested audit memo structure

- Paragraph 1: What was dangerous in the original script and why the high validation score was not trustworthy.
- Paragraph 2: What you changed and how it improves leakage control, validation, auditability, and model/policy separation.
- Limitations: one thing you would not trust yet and one thing you would improve with more time.
- Monitoring: mention at least two fairness or stability slices you would monitor after deployment, such as crop type, district, PM-Kisan status, landholding band, or irrigation type.